Московский авиационный институт
(национальный исследовательский университет)

Институт №8 «Компьютерные науки и прикладная математика»

Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №2 по курсу «Информационный поиск»

Студент:	Е. Р. Николаева
Преподаватель:	А. А. Кухтичев
Группа:	М8О-401Б-22
Дата:	
Оценка:	
Подпись:	

Москва, 2026

Лабораторная работа №2 «Поисковый робот»

Необходимо написать парсер на любом языке программирования.:

- Написать поисковый робот — компоненты обкачки документов, используя любой язык программирования; Данные для робота в секции logic: задержка между обкачкой страницы; Любые другие данные, необходимые для реализовации логики поискового робота.
- Сохранять в базе данных (например, MongoDB) документы со следующими полями: url, нормализованный; Сохранять в базе данных (например, MongoDB) документы со следующими полями: url, нормализованный; «сырой» html-текст документа; название источника; Дата обкачки документа в формате Unix time stamp.
- Поисковый робот можно остановить в любой момент и при повторном запуске робот должен начать с того документа, с которого он остановился;
- Периодически он должен уметь переобкачивать документы, которые уже есть в базе, но только в том случае, если они изменились.

1 Описание работы

В данной лабораторной работе требовалось реализовать поискового робота, выполняющего обкачку документов из заданного источника и сохраняющего результаты в базе данных. По условию задания робот должен принимать единственный аргумент командной строки — путь к YAML-конфигу, содержащему параметры базы данных и логики обхода. Также робот должен поддерживать повторный запуск после остановки и периодический переобход ранее скачанных документов с обновлением данных только в случае изменения содержимого страницы.

2 Выбор языка программирования и среды реализации

Для реализации поискового робота был выбран язык C++. Такой выбор согласуется с общими требованиями курса, согласно которым основные компоненты поисковой системы желательно реализовывать на C/C++. Для чтения конфигурационного файла использовалась библиотека YAML, для HTTP-запросов — библиотека cURL, для хранения данных — SQLite, а для анализа HTML — библиотека `libxml2`.

- база данных хранится в одном файле и не требует отдельного серверного процесса;
- структура таблиц прозрачна и удобна для отладки;
- SQLite достаточно для хранения документов и служебного состояния обхода в рамках лабораторной работы.

3 Выбор источника данных

Для поискового робота был использован тот же тематический источник, который применялся при подготовке корпуса в ЛР1. .

В качестве веб-источника была выбрана страница датасета `WINGNUS/ACL-OCL` на платформе Hugging Face, связанная с корпусом научных публикаций по тематике обработки естественного языка. Выбор именно этого источника объясняется следующими причинами:

- он связан с уже выбранным корпусом документов;
- он позволяет получить HTML-страницы, пригодные для обкачки и сохранения;

- на страницах источника присутствуют ссылки на другие страницы того же домена, что позволяет реализовать обход ссылок.

4 Требования к роботу

Согласно постановке задачи поисковый робот должен:

- принимать путь к YAML-конфигу как единственный аргумент;
- извлекать из конфига данные для базы и параметры логики обхода;
- сохранять в базе данных нормализованный URL, сырой HTML-текст, название источника и время обхода в формате Unix time stamp;
- поддерживать остановку и повторный запуск с продолжением работы;
- периодически переобходить ранее скачанные документы и обновлять их только в случае изменения.

Все эти требования были реализованы в разработанной программе.

5 Структура конфигурационного файла

Конфигурация робота хранится в YAML-файле. Использовалась следующая структура:

```
db:
path: "robot.db"

logic:
crawl_delay_seconds: 1
recrawl_interval_seconds: 86400
request_timeout_seconds: 15
max_pages_per_run: 100
max_html_bytes: 2000000
user_agent: "MySearchRobot/1.0"
allowed_domains:
-"huggingface.co"
seed_urls:
-"https://huggingface.co/datasets/WINGNUS/ACL-OCL"
```

Секция `db` содержит параметры базы данных. В данной реализации это путь к файлу SQLite. Секция `logic` содержит параметры поведения робота:

- задержку между запросами;
- интервал повторного обхода;
- таймаут HTTP-запроса;
- ограничение по числу обрабатываемых страниц за один запуск;
- ограничение на размер HTML-документа;
- User-Agent;
- допустимые домены;
- начальные URL.

6 Общая архитектура решения

Робот был реализован как консольная программа на C++, работающая с двумя основными сущностями хранения:

- таблица `documents`, содержащая уже скачанные документы;
- таблица `frontier`, содержащая очередь обхода и текущее состояние URL.

Такое разделение позволяет независимо хранить содержимое документов и состояние процесса обхода.

1 Таблица документов

В таблице `documents` хранятся:

- нормализованный URL;
- сырой HTML-текст документа;
- название источника;
- дата обхода в формате Unix time stamp.

Дополнительно были введены служебные поля:

- `etag`;
- `last_modified`;
- `content_hash`;
- `status_code`.

Эти поля используются для повторного обхода и определения изменений документа.

2 Таблица очереди обхода

Таблица `frontier` хранит URL и их состояние. Для каждой записи сохраняются:

- `url`;
- `status`;
- `discovered_from`;
- `fail_count`;
- `last_crawled_at`;
- `next_crawl_at`;
- `created_at`;
- `picked_at`.

Используются следующие статусы:

- `pending` — URL ожидает обхода;
- `in_progress` — URL взят в обработку;
- `done` — URL успешно обработан;
- `failed` — при обработке возникла ошибка.

7 Нормализация URL и фильтрация ссылок

Поскольку робот выполняет обход HTML-страниц и извлекает из них ссылки, важным этапом является нормализация URL. В реализации использовались следующие правила:

- схема и домен приводятся к нижнему регистру;
- удаляются fragment и query-параметры;
- убирается завершающий символ '/', если он не относится к корневому пути.

Такая нормализация позволяет избежать дублирования URL, ведущих к одному и тому же ресурсу.

Дополнительно была реализована фильтрация служебных и нерелевантных ссылок. Это потребовалось из-за особенностей веб-интерфейса Hugging Face, содержащего большое количество служебных страниц и ссылок, не относящихся к полезному HTML-контенту.

Отбрасывались:

- ссылки на страницы входа и регистрации;
- служебные пути вроде `/blob/`, `/tree/`, `/discussions/`;
- ссылки с query-параметрами;
- слишком длинные URL;
- ссылки на PDF-файлы и другие не-HTML ресурсы.

8 Использование полноценного HTML-парсера

Для извлечения ссылок из HTML использовался полноценный HTML-парсер на основе `libxml2`. После построения DOM-дерева выполнялся обход узлов документа и поиск тегов `a`, содержащих атрибут `href`. Такой подход надёжнее, чем выделение ссылок с помощью регулярных выражений, поскольку HTML-разметка может быть неполной, вложенной или нестрого корректной.

Относительные ссылки приводились к абсолютному виду с использованием средств работы с URI. Это позволило корректно обрабатывать ссылки вида `/path`, `./page`, `../page`.

9 Основные этапы работы робота

При запуске робот выполняет следующие действия:

1. читает YAML-конфиг;
2. открывает или создаёт базу данных;
3. создаёт таблицы при их отсутствии;
4. добавляет начальные URL в очередь обхода;
5. переводит зависшие записи `in_progress` обратно в `pending`;
6. начинает последовательный обход URL.

Для каждого выбранного URL робот:

1. извлекает из базы сохранённые метаданные документа;
2. отправляет HTTP-запрос;
3. проверяет код ответа;
4. проверяет тип содержимого;
5. ограничивает максимальный размер скачиваемого HTML;
6. сохраняет HTML в базу данных;
7. извлекает новые ссылки;
8. добавляет их в очередь;
9. обновляет статус записи в `frontier`.

10 Поддержка остановки и повторного запуска

Одним из требований задания была возможность остановить робота в любой момент и затем продолжить работу с того места, на котором она была прервана. Это реализовано за счёт хранения состояния обхода в таблице `frontier`. Когда URL выбирается для обработки, его статус меняется на `in_progress`. После успешной обработки он переводится в `done`, а при ошибке — в `failed`.

При следующем запуске все записи, оставшиеся в состоянии `in_progress`, переводятся обратно в `pending`. Благодаря этому робот способен продолжать работу после аварийного завершения или принудительной остановки без потери состояния.

11 Периодический переобход и обнаружение изменений

Для реализации периодического переобхода в таблице очереди хранится поле `next_crawl_at`. После завершения обработки документа вычисляется время следующего обхода на основе параметра `recrawl_interval_seconds`.

Чтобы обновлять документ только в случае изменения, используются два механизма:

- условные HTTP-запросы с заголовками `If-None-Match` и `If-Modified-Since`;
- сравнение хэша HTML-текста.

Если сервер возвращает код `304 Not Modified`, документ не перезаписывается. Если приходит новый HTML, вычисляется его хэш и сравнивается с ранее сохранённым значением. Таким образом, обновление выполняется только при реальном изменении содержимого.

12 Основные фрагменты кода

Для чтения конфигурации использовался следующий фрагмент:

```
Config cfg = load_config(argv[1]);
```

Данный вызов загружает YAML-файл, содержащий параметры базы и логики обхода.

Инициализация базы данных выполняется следующим образом:

```
sqlite3* db = nullptr;
if (sqlite3_open(cfg.db_path.c_str(),&db) != SQLITE_OK) {
cerr <<"Cannot open database\n";
return 1;
}

if (!init_db(db)) {
sqlite3_close(db);
return 1;
}
```

Этот фрагмент открывает SQLite-базу и создаёт необходимые таблицы.

Добавление начальных URL в очередь обхода реализуется следующим образом:

```
seed_frontier(db,cfg.seed_urls,cfg.allowed_domains);
reset_stale_in_progress(db);
```

Здесь начальные URL помещаются в таблицу `frontier`, а зависшие задачи возвращаются в состояние `pending`.

Для обработки одного URL использовалась функция следующего вида:

```
string result = process_url(db,cfg,url);
```

Внутри неё выполняются HTTP-запрос, анализ ответа, сохранение документа, извлечение ссылок и обновление состояния очереди.

Извлечение ссылок из HTML при помощи полноценного парсера реализовано через обход DOM-дерева:

```
if (xmlStrcasecmp(cur->name,BAD_CAST "a") == 0) {
xmlChar* href = xmlGetProp(cur,BAD_CAST "href");
...
}
```

Этот фрагмент показывает, что извлечение выполняется из тегов `a` и использует атрибут `href`.

13 Результаты выполнения

В результате выполнения лабораторной работы был реализован поисковый робот, удовлетворяющий требованиям задания:

- робот принимает путь к YAML-конфигу как единственный аргумент;
- параметры базы и логики обхода задаются в конфиге;
- HTML-документы сохраняются в базе данных;
- в базе сохраняются URL, HTML, источник и дата обхода;
- поддерживается остановка и повторный запуск;
- реализован периодический переобход;
- обновление выполняется только при изменении документа.

Дополнительно робот умеет фильтровать служебные URL, пропускать не-HTML-ресурсы и ограничивать размер скачиваемых страниц, что повышает устойчивость его работы на реальном веб-источнике.

```
[90] https://discuss.huggingface.co/t/looking-for-feedback-from-pytorch-users-on-an-ml-prototyping-workflow-idea/174249 -> changed
[91] https://discuss.huggingface.co/t/missing-info-in-llm-course/174258 -> changed
[92] https://discuss.huggingface.co/t/my-space-went-off-abruptly-with-error-something-went-wrong-when-restarting-this-space/174221 -> changed
[93] https://discuss.huggingface.co/t/need-help-in-fine-tuning-of-ocr-model-at-production-grade/174196 -> changed
[94] https://discuss.huggingface.co/t/preprocessing-whatsapp-for-style-cloning-how-to-handle-session-gaps-and-multi-message-blocks/174226 -> changed
[95] https://discuss.huggingface.co/t/python-gradio-space-stuck-in-restarting/173928 -> changed
[96] https://discuss.huggingface.co/t/sleeping-spaces-are-not-restarting/174003 -> changed
[97] https://discuss.huggingface.co/t/soumitra-dutta-oxford-how-do-i-run-inference-with-a-hugging-face-model/174214 -> changed
[98] https://discuss.huggingface.co/t/space-stuck-at-starting-runtime-init-failed-curl-6-could-not-resolve-host-huggingface-co/174077 -> changed
[99] https://discuss.huggingface.co/t/streamlit-spaces-safe-to-upload-data/174242 -> changed
[100] https://discuss.huggingface.co/t/valueerror-loading-helsinki-nlp-tokenizers/174192 -> changed
Done. Processed: 100

=== DATABASE SUMMARY ===
documents count: 300
frontier count: 10325

Frontier status stats:
  done -> 300
  pending -> 10025

Documents preview:
  url: https://discuss.huggingface.co/t/valueerror-loading-helsinki-nlp-tokenizers/174192
  source: discuss.huggingface.co
  crawl_timestamp: 1773533747
  status_code: 200

  url: https://discuss.huggingface.co/t/streamlit-spaces-safe-to-upload-data/174242
  source: discuss.huggingface.co
  crawl_timestamp: 1773533745
  status_code: 200

  url: https://discuss.huggingface.co/t/space-stuck-at-starting-runtime-init-failed-curl-6-could-not-resolve-host-huggingface-co/174077
  source: discuss.huggingface.co
  crawl_timestamp: 1773533744
  status_code: 200
```

14 Выводы

В ходе выполнения лабораторной работы был реализован поисковый робот, выполняющий обкачку документов из веб-источника и сохраняющий результаты в базе данных. Робот принимает YAML-конфиг, поддерживает состояние очереди обхода, умеет продолжать работу после остановки и периодически переобходить ранее скачанные документы. В качестве базы данных была использована SQLite, что позволило реализовать хранение документов и состояния обхода без необходимости в отдельном сервере базы данных. Для извлечения ссылок из HTML был использован полноценный HTML-парсер, что повысило устойчивость решения по сравнению с упрощённым выделением ссылок регулярными выражениями. В результате был получен рабочий компонент сбора документов, пригодный для дальнейшего использования в составе поисковой системы.

Список литературы

1. Хипп Р. *Документация SQLite.* — Электронный ресурс.
2. Ben-Kiki O., Evans C., döt Net I. *YAML Ain't Markup Language (YAML) Specification.* — Электронный ресурс.
3. Стенберг Д. *Документация libcurl.* — Электронный ресурс.
4. Виллатон Д. и др. *Документация libxml2.* — Электронный ресурс.

5. Hugging Face. Dataset WINGNUS/ACL-OCL. — URL: https://huggingface.co/datasets/WINGNUS/ACL-OCL